

2 marks

a) Define Data Abstraction and list its three levels.

Ans. The process of hiding irrelevant details from the user is known as data abstraction.

Three levels of Data Abstraction :-

1. Physical level
2. Logical level
3. View level

b) List any four applications of DBMS.

Ans. 1. In banking systems for storing customer information, account details and transaction records.

2. In airline reservation systems to manage flight schedules, bookings and passenger data.

3. In educational institutions for maintaining student records, attendance and exam results.

4. In ecommerce websites to handle product information, customer data and order transactions.

c) Write queries for following:

i) Creating user

ii) Deleting user

Ans. CREATE USER username IDENTIFIED BY password;

e.g. CREATE USER Jay IDENTIFIED BY j;

d) Give syntax for creating views.

Ans. CREATE VIEW view_name AS

SELECT column 1,column 2

FROM table_name

WHERE condition;

e.g. CREATE VIEW stud AS

SELECT roll_no, name, marks

FROM student

WHERE marks > 50;

e) Define normalization. Enlist its types.

Ans. Normalization is a systematic approach to organize data within a database to reduce data redundancy and eliminate undesirable characteristics such as insertion, updation and deletion anomalies.

Types of normalization :-

1. First Normal Form (1NF)
2. Second Normal Form (2NF)
3. Third Normal Form (3NF)

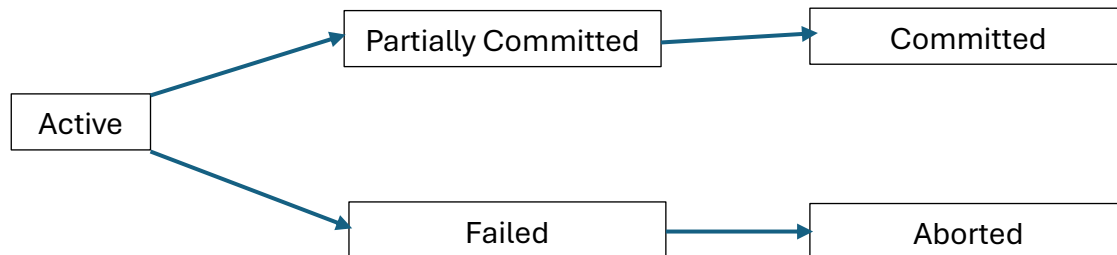
f) Enlist string function in DBMS.

Ans. 1.LENGTH(string) : Returns the number of characters in a string.

2. LOWER(string) : Converts all letters in the string to lowercase.
3. UPPER(string) : Converts all letters in the string to uppercase.
4. CONCAT(string1, string2) : Concatenates two strings.
5. TRIM(string) : Removes leading and trailing spaces.
6. INITCAP(string) : Converts the first letter of the string to uppercase and rest to lowercase.
7. INSTR(string) : Returns the position of the first occurrence of a substring.
8. SUBSTR(string, starting_position, length) : Extracts part of the string.

g) Draw states of transaction diagram in DBMS.

Ans.



4 marks

a) Explain grant and revoke command with syntax and example.

Ans. 1. GRANT Command : Used to give specific privileges to a user.

Syntax : GRANT privilege_name ON object_name TO user_name;

Example : GRANT SELECT, UPDATE ON employees TO scott;

2. REVOKE Command : Used to remove privileges granted earlier.

Syntax : REVOKE privilege_name ON object_name FROM user_name;

Example : REVOKE UPDATE ON employees FROM scott;

b) Explain ACID properties of transaction.

Ans. ACID properties are a set of key principles that ensure reliability and consistency of database transactions.

1. Atomicity : A transaction is all or nothing. Either all operations in a transaction are executed or none are.
e.g. Transferring money between accounts, either both debit and credit happens or neither.

2. Consistency : A transaction takes the database from one valid state to another, maintaining all integrity constraints.
e.g. After a transaction, total account balances remain correct.

3. Isolation : Transactions are executed independently of others. Intermediate states of a transaction are not visible to other transaction.

e.g. Two users updating the same account will not interfere with each other.

4. Durability : Once a transaction is committed, its changes are permanent, even in case of a system failure.

e.g. Money transferred is permanently recorded even if the system crashes immediately after.

c) Explain Arithmetic function with syntax and example.

Ans.

Function	Description	Example SQL	Result
ABS(n)	Returns the absolute value of n.	ABS(-15.7)	15.7
MOD(m, n)	Returns the remainder of m divided by n (Modulo).	MOD(10, 3)	1
POWER(m,n)	Returns m raised to the power of n (m^n).	POWER(4, 3)	64
SQRT(n)	Returns the square root of n.	SQRT(25)	5
ROUND(n,[d])	Rounds n to d decimal places (defaults to 0).	ROUND(15.789, 1)	15.8

d) Explain PL/SQL block structure with example.

Ans. DECLARE

--Declaration section (optional)

BEGIN

--Execution section (mandatory)

EXCEPTION

--Exception handling section (optional)

Declaration section : All the variables, constants, cursors and user-defined exceptions are declared in this section.

Execution sections : All the executable statements are in this section.

Exception section : Error handling/ exceptions are defined in this section.

a) Explain create and alter command with syntax and example.

Ans. 1. CREATE Command : Used to create a new table or database object.

Syntax :

CREATE TABLE table_name (column1 datatype, column2 datatype);

Example :

CREATE TABLE employees (emp_id NUMBER, emp_name VARCHAR2(30), salary NUMBER);

2. ALTER Command : Used to modify an existing table structure.

Syntax :

ALTER TABLE table_name ADD column_name datatype;

Example : ALTER TABLE employees ADD department VARCHAR2(20);

b) What is database backup explain types of failure and causes of failure.

Ans. A **database backup** is a copy of your database's data, structure (schema), and configuration files, stored separately from the live database. Its primary purpose is to allow for **data recovery** in case the original data is lost, corrupted, or becomes inaccessible due to a system failure or disaster.

Types of Database Failure

Database failures can be broadly classified based on the scope and cause of the problem:

1. Transaction Failure:

- **Scope:** Limited to one or a few transactions.
- **Cause:** A transaction cannot complete due to an internal **logical error** (e.g., a bug in the code) or a **system error** (e.g., **deadlock** between concurrent transactions, or insufficient resources).

2. System Crash (Soft Failure):

- **Scope:** Affects the entire system and all active transactions.
- **Cause:** A sudden, external interruption that stops the system abruptly. The data on stable storage (disk) is usually fine, but data in **volatile memory** (like RAM or system buffers) is lost.
- **Examples:** Power outage, operating system (OS) crash, or a software error in the database management system (DBMS) itself.

3. Media Failure (Catastrophic Failure):

- **Scope:** Loss or corruption of the non-volatile storage where the primary database files are stored.
- **Cause:** Physical damage to the storage device.
- **Examples:** Hard disk crash, formation of **bad sectors** on the disk, or a natural disaster (like fire or flood) destroying the physical data center.

Causes of Failure :-

1. Human Error : The most common cause of data loss.

Accidental Deletion/Update : A user or administrator mistakenly drops a table or runs an incorrect SQL query.

Misconfiguration : Improperly setting access controls, backup schedules, or system parameters.

2. Hardware & Infrastructure : Physical components breaking down or external events.

Disk/Drive Failure : A server's hard drive fails, often resulting in media failure.

Power Fluctuation : Sudden power loss or surges can corrupt data being written to the disk.

Network Failure : Interruption in connectivity can cause distributed transactions to fail.

3. Software & Security : Issues arising from code or malicious activity.

DBMS or OS Bugs: Errors in the database software or operating system causing corruption or crashes.

Data Corruption: Silent errors, often due to faulty hardware or unhandled exceptions, leading to unreadable or inconsistent data blocks.

Cyberattacks (Ransomware): Malicious software encrypts or deletes the database, requiring a clean backup to recover.

c) Explain predefined and user defined exception handling with the help of example.

Ans. 1. Predefined (standard) exceptions : These are automatically raised by oracle for common runtime errors.

E.g. ZERO_DIVIDE : Raised when a number is divide by zero.

NO_DATA_FOUND : Raised when a SELECT query returns no rows.

TOO_MANY_ROWS : Raised when a SELECT query returns multiple rows.

VALUE_ERROR : Raised when an invalid conversion or assignment occurs.

Example :-

```
DECLARE
```

```
v_num1 NUMBER := 10;
```

```
v_num2 NUMBER := 0;
```

```
v_result NUMBER;
```

```
BEGIN
```

```
v_result := num1/num2;
```

```
EXCEPTION
```

```
WHEN ZERO_DIVIDE THEN
```

```
Dbms_output.put_line('Error cannot divide by zero');
```

```
END;
```

```
/
```

2. User-defined exceptions : Theses are explicitly declared by the programmer for specific business logic errors. Declared using EXCEPTION keyword. Raised using RAISE keyword.

Example :

```
DECLARE
```

```
v_age NUMBER := 15;
```

```
age_exception EXCEPTION;
```

```
BEGIN
```

```
IF v_age < 18 THEN
```

```
RAISE age_exception;
```

```
END IF;
```

```
EXCEPTION
```

```
WHEN age_exception THEN
```

```
dbms_output.put_line('Error : Age must be 18 or above');
```

```
END;
```

```
/
```

d) Write and explain the syntax for creating and dropping sequences with example.

Ans. Used to generate unique numeric values automatically.

Syntax to create :

```
CREATE SEQUENCE seq_name
```

```
START WITH n
```

```
INCREMENT BY m;
```

Example :

```
CREATE SEQUENCE emp_seq START WITH 1 INCREMENT BY 1;
```

Syntax to drop :

```
DROP SEQUENCE seq_name;
```

Example :

```
DROP SEQUENCE emp_seq;
```

a) List and explain DML commands with syntax and example.

Ans. 1. SELECT

The SELECT command is used to **retrieve** data from one or more tables in the database. It is the most frequently used SQL command.

Syntax :

```
SELECT column1, column2, ...
```

```
FROM table_name
```

```
WHERE condition;
```

Example :

```
SELECT e_name, salary
```

```
FROM employees
```

```
WHERE department = 'Sales';
```

2. INSERT

The INSERT command is used to **add** new rows (records) into a table.

Syntax :

```
INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
```

Example :

```
INSERT INTO employees (employee_id, employee_name, department) VALUES (101, 'Jane Doe', 'Marketing');
```

3. UPDATE

The UPDATE command is used to **modify** existing data within the rows of a table.

Syntax :

```
UPDATE table_name SET column = new_value WHERE condition;
```

Example :

```
UPDATE employees
```

```
SET salary = salary * 1.10
```

```
WHERE department = 'Marketing';
```

4. DELETE

The DELETE command is used to **remove** existing rows (records) from a table.

Syntax

```
DELETE FROM table_name
```

```
WHERE condition;
```

Example

```
DELETE FROM employees
```

```
WHERE department = 'HR';
```

b) Draw overall structure of DBMS.

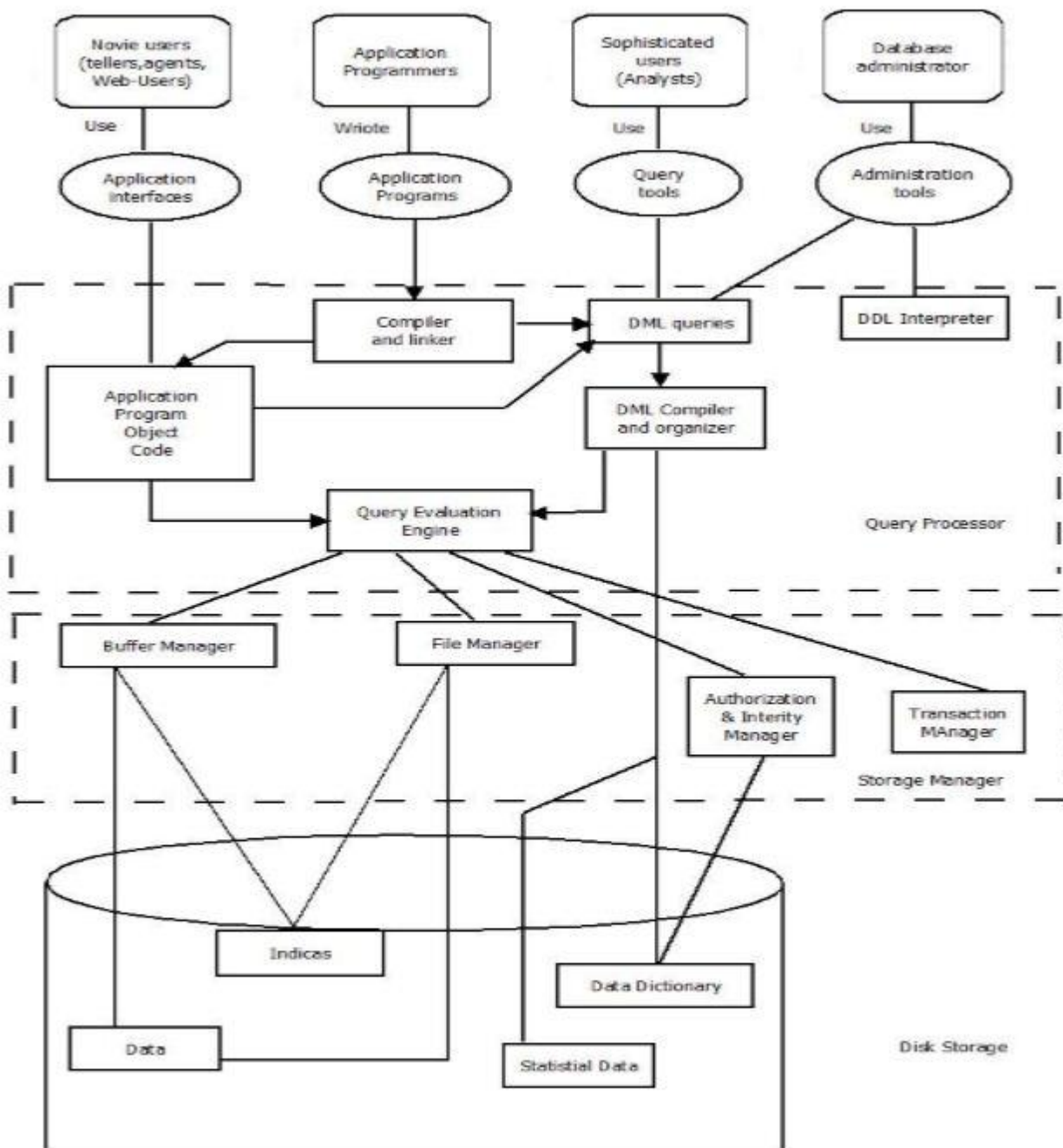


Fig1. Database System Architecture.

c) List control structure in PL/SQL explain conditional control structure with syntax and example.

Ans. Types:

1. Conditional Control – IF , IF-ELSE, IF-ELSIF statements
2. Iterative Control – LOOP, FOR, WHILE
3. Sequential Control – GOTO, NULL

Conditional control structures execute a specific sequence of statements based on whether a given condition evaluates to TRUE or FALSE. They allow your code to make decisions.

Syntax:

IF condition THEN

statements;

ELSE

statements;

END IF;

Example:

IF salary > 5000 THEN

DBMS_OUTPUT.PUT_LINE('High Salary');

ELSE

DBMS_OUTPUT.PUT_LINE('Low Salary');

END IF;

d) List and explain advantages of DBMS over file processing system.

- Ans.** 1. Reduced data redundancy : Data is stored in a centralized way, avoiding unnecessary duplication.
2. Data consistency : Changes made in one place are reflected everywhere.
3. Improved data security : Access rights can be granted or restricted to specific users.
4. Better data integrity : Ensures accuracy and validity of data through constraints and rules.

e) Explain aggregate function with example.

Ans. 1. COUNT() : Returns the number of rows in a table or rows matching a condition.

e.g. SELECT COUNT(*) FROM employees;

2. SUM() : Returns the total sum of a numeric column.

e.g. SELECT SUM(salary) FROM employees;

3. AVG() : Returns the average value of a numeric column.

e.g. SELECT AVG(salary) FROM employees;

4. MAX() : Returns the maximum value in a column.

e.g. SELECT MAX(salary) FROM employees;

6 marks

a) Draw ER diagram of Railway Management System.

Ans.

b) Write PL/SQL program to identify largest of three numbers. (accept numbers from user.)

Ans. DECLARE

a NUMBER := 25;

b NUMBER := 40;

c NUMBER := 30;

largest NUMBER ;

```

BEGIN
IF (a>b) AND (a>c) THEN
largest := a;
ELSIF (b>a) AND (b>c) THEN
largest := b;
ELSE
largest := c;
END IF;
dbms_output.put_line('Largest number is :- ' || largest);
END;
/

```

c) Explain joins in SQL with examples.

Ans. **Joins** are fundamental operations in SQL used to combine rows from two or more tables based on a related column between them. This is necessary because data in a relational database is typically spread across multiple tables to maintain efficiency and minimize redundancy.

The related column is usually a **Primary Key** in one table and a **Foreign Key** in the other.

There are four primary types of joins: INNER, LEFT (OUTER), RIGHT (OUTER), and FULL (OUTER).

1. INNER JOIN

The INNER JOIN returns only the rows that have **matching values** in **both tables**. Any rows that do not have a match in the other table are excluded from the result set.

Syntax :

```

SELECT columns
FROM TableA
INNER JOIN TableB
ON TableA.key_column = TableB.key_column;

```

Example :

```

SELECT E.Name, D.DepartmentName
FROM Employees E
INNER JOIN Departments D
ON E.DepartmentID = D.DepartmentID;

```

2. LEFT JOIN (or LEFT OUTER JOIN)

The LEFT JOIN returns **all rows** from the **left table** (the first table specified) and the **matching rows** from the **right table**. If there is no match in the right table, NULL is returned for the right table's columns.

Syntax :

```

SELECT columns
FROM TableA

```

LEFT JOIN TableB

ON TableA.key_column = TableB.key_column;

Example :

SELECT E.Name, D.DepartmentName

FROM Employees E

LEFT JOIN Departments D

ON E.DepartmentID = D.DepartmentID;

3. RIGHT JOIN (or RIGHT OUTER JOIN)

The RIGHT JOIN is the mirror image of the LEFT JOIN. It returns **all rows** from the **right table** and the **matching rows** from the **left table**. If there is no match in the left table, NULL is returned for the left table's columns.

Syntax :

SELECT columns

FROM TableA

RIGHT JOIN TableB

ON TableA.key_column = TableB.key_column;

Example :

SELECT E.Name, D.DepartmentName

FROM Employees E

RIGHT JOIN Departments D

ON E.DepartmentID = D.DepartmentID;

4. FULL JOIN (or FULL OUTER JOIN)

The FULL JOIN returns **all rows** when there is a match in **one of the tables**. It combines the results of both LEFT and RIGHT joins. If a row in one table does not have a match in the other, NULL is returned for the columns of the non-matching table.

Syntax :

SELECT columns

FROM TableA

FULL JOIN TableB

ON TableA.key_column = TableB.key_column;

Example :

SELECT E.Name, D.DepartmentName

FROM Employees E

FULL JOIN Departments D

ON E.DepartmentID = D.DepartmentID;

a) Consider the following schema.

Student (Name, Mark, Age, Place, Phone, Birthdate) Write SQL queries for following:

i) To list name of student who do not have phone number.

ii) To list student from Nashik and Pune.

iii) To change mark of Monika to 88 instead of 80.

iv) To list student from Anit's age group.

Ans. i. SELECT name FROM Student WHERE phone = 'NULL';

ii. SELECT name FROM Student WHERE place = 'Nashik' OR place = 'Pune';

iii. ALTER TABLE Student SET mark = 88 WHERE name = 'Monika';

iv. SELECT name FROM Student WHERE age = (SELECT age FROM Student WHERE name = 'Anit');

b) Write PL/SQL program to find even of odd numbers.

Ans. DECLARE

v_num NUMBER := &input_number;

BEGIN

IF MOD(v_num,2) = 0 THEN

dbms_output.put_line('The entered number is Even.');

ELSE

dbms_output.put_line('The entered number is Odd.');

END IF;

END;

/

c) What is data integrity constraints explain types of data integrity constraints with example.

Ans. **Data Integrity Constraints** are rules defined on columns in a table to enforce the accuracy, consistency, and reliability of the data stored within a relational database.

They prevent the entry of invalid data into the tables, ensuring that the information remains correct, regardless of the operations performed on it (like INSERT, UPDATE, or DELETE).

1. **PRIMARY KEY** : A column or set of columns that uniquely identifies each row. It combines UNIQUE and NOT NULL rules.

e.g. EmployeeID is a Primary Key. Two employees cannot have the same EmployeeID.

Example :

CREATE TABLE Employees (EmployeeID INT PRIMARY KEY, Name VARCHAR(100) NOT NULL);

2. **FOREIGN KEY** : A column in one table that points to the Primary Key of another table.

e.g. The DepartmentID in the Employees table must match an existing ID in the Departments table.

Example :

CREATE TABLE Departments (ID INT PRIMARY KEY, Name VARCHAR(100));

```
CREATE TABLE Employees ( EmployeeID INT PRIMARY KEY, Name VARCHAR(100), DepartmentID INT,  
FOREIGN KEY (DepartmentID) REFERENCES Departments(ID) );
```

3. Domain Integrity

This constraint ensures that all values in a column are drawn from a valid set of values or a valid data format.

Constraint	Description	Example
NOT NULL	Ensures that a column cannot accept NULL values.	EmployeeName must always be provided.
CHECK	Defines a condition that all values in a column must satisfy.	Age must be greater than or equal to 18.
DEFAULT	Automatically assigns a default value if no value is explicitly provided during insertion.	If Status is not specified, it defaults to 'Active'.

Example :

```
CREATE TABLE Student (roll_no INT, name VARCHAR(50) NOT NULL, phone INT NULL, city VARCHAR(40)  
DEFAULT = 'Panvel', age INT, check(age>18));
```